

Mock Final Practice (Spring 2026) — Answer Key

1 Machine Representation (16 pts, 2 pts each)

A. `addq $4, %rbx` adds 4 (64-bit immediate) to the 64-bit register `%rbx`.

Correct answers: **3** (`long, v+=4`), **4** (`unsigned long, v+=4`), **6** (`int*, v++`)

Rationale: The suffix `q` indicates the instruction computes with 8-byte values. So variable `v` must be a type of size 8-bytes, which `long`, `unsigned long` and any pointer type could satisfy. For choice 5, it corresponds to `addq $16, %rbx`. For 7, it corresponds to `addq $32, %rbx`. For 8, it corresponds to `addq $8, %rbx`.

B. `movq (%rdi, %rsi, 8), %rax` reads 8 bytes from memory at address `%rdi + %rsi × 8` and stores them in `%rax`. Since `%rdi` stores the base address, it must be of a pointer type. Since `%rsi` is a 8-byte register storing index value, it must be of an 8-byte integer type (aka `long` or `unsigned long`). Finally since the instruction reads 8 bytes and the stride (or scale) of access is 8, the instruction is accessing array of 8-byte elements. Thus, 2 is the correct answer.

Correct answer: **2** (`long*` and `long`).

C. Instructions that modify `%rsp`:

Correct answers: **1** (`ret, pops 8B ⇒ %rsp+=8`), **2** (`pushq, %rsp-=8`), **3** (`popq, %rsp+=8`), **4** (`call, pushes return address ⇒ %rsp-=8`).

`movq %rax, (%rsp)` writes to the memory at `%rsp` but does not change `%rsp` itself.

D. Instructions that read from or write to memory:

Correct answers: **1** (`ret` reads return address from stack), **2** (`pushq` writes to stack), **3** (`popq` reads from stack), **4** (`call` writes return address to stack).

`addq %rax, %rbx` operates on registers only.

E. Dereferencing `%rsi` (reading from or writing to the address stored in `%rsi`):

Correct answers: **2** (`movq (%rsi), %rax` reads from `*y`), **4** (`subq %rax, (%rsi)` writes to `*y`). `leaq` computes an address without accessing memory. `movq %rsi, %rax` copies the pointer value, not the pointed-to value.

F. `foo(128)` recurses 128 times; peak call depth is 129 frames (`foo(128)`, `foo(127)`, ..., `foo(0)`). Each frame needs at least 8 bytes for the return address, giving $129 \times 8 = 1032$ bytes minimum.

Correct answer: **1** (8096 bytes). 1024 bytes ($= 128 \times 8$) is one call short.

G. Segmentation faults:

Correct answer: **3**. Dereferencing NULL (address 0) always faults because page 0 is never mapped.

Statement 1 is false (out-of-bounds may hit valid mapped memory). Statement 2 is false (reads can also fault). Statement 4 is false (pointer arithmetic itself is legal).

H. Local `int a[16]`: $16 \times 4 = 64$ bytes allocated on the stack.

Correct answers: **1** (stack, not heap), **3** (`subq $64, %rsp` allocates; `addq $64, %rsp` deallocates — the stack grows *downward*).

2 Basic C (10 pts, 2 pts each)

A. `p = (int *)5` treats the integer value 5 as a pointer address. Dereferencing address 5 accesses an unmapped page.

Correct answer: **2** (segmentation fault).

B. `'d'` is at 0-based index 3 in `"abcdef"`.

Correct answers: **2** (`v = *(s+3)`), **3** (`v = s[3]`).

Both evaluate to the character at offset 3. Option 1 and 5 assign a pointer to a `char` (which will result in compilation error unless explicitly type casted). Option 4 (`* (++s)`) increments `s` first, yielding `s[1]='b'`.

C. `a = {0, 0, -1, -1, -1}`, `p = (int*)(a+1)`. On a little-endian 64-bit system, reading 4 bytes from `a[1]` gives bytes `0x00, 0xFF, 0xFF, 0xFF` $\Rightarrow$ `0xFFFFFFFF00` = -256 (signed).

Correct answer: **2** (a negative number). Value is -256 .

D. “Computes 0 iff the i -th bit from the left (MSB=bit 0) is zero.” Need a mask with exactly bit $31 - i$ set from the right.

Correct answer: **3** (`x & (1 << (31-i))`).

Option 1 only works if i is zero.

E. “Sets the i -th bit from the left to 1.” Need to OR with 2^{31-i} .

Correct answer: **3** (`x |= (1 << (31-i))`).

3 C to Assembly (20 pts)

A. (2 pts) Function arguments follow x86-64 calling convention: first argument in `%rdi`, second in `%rsi`.

Answer: `a` is in `%rdi`; `b` is in `%rsi`.

B. (4 pts) C-to-assembly mapping for `swap`:

C line	Assembly lines
5 (<code>tmp = *a</code>)	2: <code>movq (%rdi), %rax</code>
6 (<code>*a = *b</code>)	3–4: <code>movq (%rsi), %rdx; movq %rdx, (%rdi)</code>
7 (<code>*b = tmp</code>)	5: <code>movq %rax, (%rsi)</code>

C. (2 pts) In `reverse_array` (labeled `reverse_str` in assembly):

Answer: `s` is in `%rbp`; `e` is in `%rbx`.

This can be inferred from line 12 and 11, which set up the first argument (storing `a[s]` in `%rdi`) and second argument (`a[e]` in `%rsi`) prior to making the function call `call swap`.

D. (1 pt) Loop condition `s < e`:

Answer: Line 9 (`cmpq %rbx, %rbp`) computes `s - e` to set flags; line 10 (`jge .L6`) exits if `s ≥ e`.

E. (1 pt) Arguments copied for `swap`:

Answer: Lines 11–12: `movq (%r12,%rbx), %rsi` loads `a[e]` into second argument; `movq (%r12,%rbp), %rdi` loads `a[s]` into first argument.

Note: this is the bug — `a[s]` and `a[e]` are the *values*, but `swap` expects addresses as arguments (see H/I).

F. (2 pts) `%r12` is set at line 5: `movq %rdi, %r12`. `%rdi` holds the first argument.

Answer: 1 (a).

G. (4 pts) Starting `%rsp = 0x00007FFFFFFFFF20` before line 2. Lines 2, 3, 4 are three `pushq` instructions, each decrementing `%rsp` by 8:

$$\%rsp = 0x00007FFFFFFFFF20 - 24 = 0x00007FFFFFFFFF08$$

Line 13 is `call swap`, which pushes the 8-byte return address:

$$\%rsp \text{ immediately after} = 0x00007FFFFFFFFF08 - 8 = 0x00007FFFFFFFFF00$$

Answers: Before line 13: `0x00007FFFFFFFFF08`; after: `0x00007FFFFFFFFF00`.

H. (2 pts) The bug: line 6 calls `swap((long *)a[s], (long *)a[e])`. When `s = 0`, `a[s] = a[0] = 1` (the array value), which is then cast to a pointer. So `%rdi = 1`.

Answer: `%rdi = 1`.

I. (2 pts) Fix: pass addresses, not values.

Correction: Change line 6 from `swap((long *)a[s], (long *)a[e])` to `swap(&a[s], &a[e])`.

4 More C Programming (6 pts)

(3 pts) **hex2digit:**

```
char hex2digit(char c) {
    if (c >= '0' && c <= '9')
        return c - '0';           // '0'=48, so c-'0' gives 0..9
    return c - 'a' + 10;          // 'a'=97, so c-'a'+10 gives 10..15
}
```

(3 pts) **hex2int:**

```
int hex2int(char *s) {
    assert(strlen(s) == 8);
    int result = 0;
    for (int i = 0; i < 8; i++) {
        result = (result << 4) | hex2digit(s[i]);
    }
    return result;
    // Reinterprets the 32-bit bit pattern as 2's complement int.
    // "ffffffff" -> 0xFFFFFFFF -> (int)-1.  "0000000f" -> 15.
}
```

5 Combinatorial Logic (12 pts)

A. (2 pts) Prove $X \cdot Y + X \cdot \bar{Y} = X$:

$$\begin{aligned}
 X \cdot Y + X \cdot \bar{Y} &= X \cdot (Y + \bar{Y}) && \text{(distributive law)} \\
 &= X \cdot 1 && \text{(complement law)} \\
 &= X && \text{(identity law)}
 \end{aligned}$$

Alternatively, you can prove by enumerating the truth table:

X	Y	$X \cdot Y + X \cdot \bar{Y}$
0	0	0
0	1	0
1	0	1
1	1	1

Because the third column is identical to the column X , the equality holds.

B. (5 pts) 1-bit adder with inputs a_i, b_i, c_i^{in} and outputs s_i, c_i^{out} .

(1) Truth table:

a_i	b_i	c_i^{in}	s_i	c_i^{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

(2) Sum-of-products for s_i (rows where $s_i = 1$: 001, 010, 100, 111):

$$s_i = \bar{a}_i \cdot \bar{b}_i \cdot c_i^{in} + \bar{a}_i \cdot b_i \cdot \bar{c}_i^{in} + a_i \cdot \bar{b}_i \cdot \bar{c}_i^{in} + a_i \cdot b_i \cdot c_i^{in}$$

(3) Sum-of-products for c_i^{out} (rows where $c_i^{out} = 1$: 011, 101, 110, 111):

$$c_i^{out} = \bar{a}_i \cdot b_i \cdot c_i^{in} + a_i \cdot \bar{b}_i \cdot c_i^{in} + a_i \cdot b_i \cdot \bar{c}_i^{in} + a_i \cdot b_i \cdot c_i^{in}$$

(4) Prove equivalence to $a_i \cdot c_i^{in} + b_i \cdot c_i^{in} + a_i \cdot b_i$:

$$\begin{aligned}
 c_i^{out} &= \bar{a}_i \cdot b_i \cdot c_i^{in} + a_i \cdot \bar{b}_i \cdot c_i^{in} + a_i \cdot b_i \cdot \bar{c}_i^{in} + a_i \cdot b_i \cdot c_i^{in} \\
 &= \bar{a}_i \cdot b_i \cdot c_i^{in} + a_i \cdot b_i \cdot c_i^{in} + a_i \cdot \bar{b}_i \cdot c_i^{in} + a_i \cdot b_i \cdot c_i^{in} + a_i \cdot b_i \cdot \bar{c}_i^{in} + a_i \cdot b_i \cdot c_i^{in} \\
 &= b_i \cdot c_i^{in} (\bar{a}_i + a_i) + a_i \cdot c_i^{in} (\bar{b}_i + b_i) + a_i \cdot b_i (\bar{c}_i^{in} + c_i^{in}) \\
 &= b_i \cdot c_i^{in} + a_i \cdot c_i^{in} + a_i \cdot b_i \quad \checkmark
 \end{aligned}$$

In the above derivation, we use the rule $X + X = X$ to duplicate the term $a_i \cdot b_i \cdot c_i^{in}$ twice.

Alternatively, you can use the brute force method of proving equivalence by showing that the truth table of expression $\overline{a_i} \cdot b_i \cdot c_i^{in} + a_i \cdot \overline{b_i} \cdot c_i^{in} + a_i \cdot b_i \cdot \overline{c_i^{in}} + a_i \cdot b_i \cdot c_i^{in}$ matches that of expression $b_i \cdot c_i^{in} + a_i \cdot c_i^{in} + a_i \cdot b_i$

C. (5 pts) Four-bit ripple-carry adder: chain four 1-bit adders with carry propagating left. Suppose the four 1-bit adders are $adder_3 \dots adder_0$. Given two four-bit numbers $a_3a_2a_1a_0$, $b_3b_2b_1b_0$, connect a_i, b_i to the a/b input ports of $adder_i$. Set $c_0^{in} = 0$ and connect it to the carry-in port of $adder_0$. Connect the carry-out of $adder_i$ to the carry-in port of $adder_{i+1}$, aka $c_{i+1}^{in} = c_i^{out}$. The overflow flag $of = c_3^{out}$ (carry out of the MSB adder).

6 FSM (10 pts)

State encoding: $s_1s_0 = 00 \Rightarrow \text{pos } 0$ (rightmost); $01 \Rightarrow \text{pos } 1$; $10 \Rightarrow \text{pos } 2$; $11 \Rightarrow \text{pos } 3$ (leftmost). Joystick: $x = 0$ (push left) increases position by 1 (clamped at 3); $x = 1$ (push right) decreases position by 1 (clamped at 0).

A. (3 pts) Trace for input 0, 1, 1, 1 starting at $s_1s_0 = 00$:

Step	x	s_1	s_0	y_3	y_2	y_1	y_0
1	0	0	0	0	0	0	1
2	1	0	1	0	0	1	0
3	1	0	0	0	0	0	1
4	1	0	0	0	0	0	1

Step 1: $x = 0$ from pos 0 $\rightarrow$ pos 1 ($s_1s_0 = 01$). Step 2: $x = 1$ from pos 1 $\rightarrow$ pos 0 ($s_1s_0 = 00$). Steps 3–4: $x = 1$ from pos 0, already rightmost, stays at 00.

B. (4 pts) Full truth table (note: $y_i = 1$ iff current state $s_1s_0 = i$):

x	s_1	s_0	$next_s_1$	$next_s_0$	y_3	y_2	y_1	y_0
0	0	0	0	1	0	0	0	1
0	0	1	1	0	0	0	1	0
0	1	0	1	1	0	1	0	0
0	1	1	1	1	1	0	0	0
1	0	0	0	0	0	0	0	1
1	0	1	0	0	0	0	1	0
1	1	0	0	1	0	1	0	0
1	1	1	1	0	1	0	0	0

C. (2 pts) Sum-of-products for $next_s_1$ (rows 001,010,011,111 where $next_s_1 = 1$):

$$next_s_1 = \overline{x} \cdot \overline{s_1} \cdot s_0 + \overline{x} \cdot s_1 \cdot \overline{s_0} + \overline{x} \cdot s_1 \cdot s_0 + x \cdot s_1 \cdot s_0$$

D. (1 pt) We note that $y_k = 1$ iff the state binary value s_1s_0 equals k . This is exactly a **2-to-4 decoder** with inputs (s_1, s_0) and outputs (y_3, y_2, y_1, y_0) .

7 Non-Pipelined and Pipelined CPU (12 pts)

Refer to the non-pipelined RISC-V CPU figure ([simple-cpu.pdf](#)).

A. (4 pts) For `ld x5, 40(x6)` ($x5 = \text{Memory}[x6 + 40]$):

- Instruction[11-7]** is the `rd` field, which encodes the destination register $x5$. Register 5 in 5-bit binary: **00101**.
- ALUSrc**: The ALU must add the base register ($x6$) to the immediate (40). The second ALU input should be the sign-extended immediate $\Rightarrow$ select input port **1**.
- Data value at input port 1** of the ALU Mux: this duplicates the previous question.
- MemtoReg**: For a load, the data written to the register file comes from data memory (not the ALU result) $\Rightarrow$ select input port **1**.

B. (4 pts) Latencies: ①=200 ps, ②=100 ps, ③=200 ps, ④=200 ps, ⑤=100 ps.

(1) 5-stage pipeline (each component = one stage):

- Clock cycle = slowest stage = $\max(200, 100, 200, 200, 100) = \mathbf{200}$ ps.
- Instruction latency = $5 \times 200 = \mathbf{1000}$ ps.
- Throughput = $1/200 \text{ ps} = \mathbf{5 \times 10^9}$ instructions/sec

(2) 3-stage pipeline (Stage 1: ①+②; Stage 2: ③; Stage 3: ④+⑤):

- Stage latencies: 300 ps, 200 ps, 300 ps.
- Clock cycle = $\max(300, 200, 300) = \mathbf{300}$ ps.
- Instruction latency = $3 \times 300 = \mathbf{900}$ ps.
- Throughput = $1/300 \text{ ps} \approx \mathbf{3.33 \times 10^9}$ instructions/sec.

C. (4 pts) Instruction sequence: ① `ld x1, 16(x2)` ② `add x3, x1, x4` ③ `sd x5, 8(x2)`

(i) (1 pt) Hazard: RAW on register $x1$ between instructions ① and ②. ① writes $x1$ in WB; ② reads $x1$ in ID. With same-cycle read/write: ②'s ID can occur in the same cycle as ①'s WB $\Rightarrow$ 2-cycle stall needed. No hazard between ② and ③ (they share no registers).

(ii) (3 pts) Pipeline diagram:

Instruction	cy 1	cy 2	cy 3	cy 4	cy 5	cy 6	cy 7	cy 8	cy 9	cy 10	cy 11
① <code>ld x1, 16(x2)</code>	IF	ID	EX	MEM	WB						
no-op		IF	ID	EX	MEM	WB					
no-op			IF	ID	EX	MEM	WB				
② <code>add x3, x1, x4</code>				IF	ID	EX	MEM	WB			
③ <code>sd x5, 8(x2)</code>					IF	ID	EX	MEM	WB		

Verification: ① WB at cycle 5; ② ID at cycle 5 — reads the value $x1$ being written in the same cycle. ✓

8 Memory Hierarchy (8 pts)

Cache parameters: 32-bit addresses, 1 KB total size, 64-byte cache lines.

A. (2 pts) Number of cache lines = $1024/64 = 16$.

B. (3 pts) Direct-mapped cache (16 lines):

- Byte offset: $\log_2(64) = 6$ bits (bits 5:0).
- Index: $\log_2(16) = 4$ bits (bits 9:6).
- Tag: $32 - 6 - 4 = 22$ bits (bits 31:10).
- Address layout: [tag:22] [index:4] [offset:6].

For 0xab345f78: bits 9:6 of 0x5f78: 0x5f78 in binary ends with ...0101 1111 0111 1000. Bits 9:6 = $(1101)_2 = 13$. (Cache line index = 13.)

Number of cacheline-aligned addresses mapping to the same index: $2^{32}/(64 \times 16) = 2^{22}$.

The **tag** distinguishes which of those 2^{22} addresses is actually stored at that cache line.

C. (3 pts) 2-way set-associative (8 sets):

- Byte offset: 6 bits (bits 5:0).
- Set index: $\log_2(8) = 3$ bits (bits 8:6).
- Tag: $32 - 6 - 3 = 23$ bits (bits 31:9).

For 0xab345f78: bits 8:6 of 0x78 = $0x78 \gg 6 \bmod 8$. $0x78 = 0111\ 1000_2$; bits 8:6 = $(101)_2 = 5$ (set index = 5).

Cacheline-aligned addresses mapping to the same set: $2^{32}/(64 \times 8) = 2^{23}$

9 Virtual Memory (6 pts)

A. (2 pts) True statements about VA→PA translation:

Correct answers: **2** (done by the MMU hardware), **3** (done at the granularity of pages).

The OS sets up page tables but the *MMU* performs each translation at runtime. Translation maps virtual *page numbers* to physical *page numbers* (granularity = page), not bytes or cachelines.

B. (2 pts) True statements:

Correct answers: **1** (each process has its own page table), **3** (64-bit x86 uses 4-level page tables).

C. (2 pts) 20-bit addresses, 2-level page table, 256-byte pages, 4-byte PTEs.

Each page holds $256/4 = 2^6 = 64$ PTEs, so each level uses 6 bits of index.

1. Page offset = $\log_2(256) = 8$ bits (bits 7:0). Remaining $20 - 8 = 12$ bits split evenly: L0 (top-level) = 6 bits (bits 19:14), L1 (second-level) = 6 bits (bits 13:8). Layout: [L0:6] [L1:6] [offset:8].
2. Maximum memory = 2^{20} bytes (aka 1MB)
Page tables: 1 L0 table + $2^6 = 64$ L1 tables. Since each L1 or L0 table is exactly one page in size (256 bytes), the total size of L0+L1 tables is: $1 \times 256 + 64 \times 256 = 256 \times 65$ bytes.
3. Addresses $0x00000-0x0001f$ (32 bytes) all lie in page 0 (L0 index = 0, L1 index = 0). Only one L0 table (256 B) and one L1 table (256 B) are needed = **512** bytes.